

University Course Management System

Part 2: Formalizing the UML Class Diagram

Overview

Part 1 asked you to implement Student, Professor, Course, and Department, and to sketch a UML class diagram of how they relate. Part 2 has no new classes and no new methods to write — `main()` simply prints "All done!". Instead, the deliverable this time is the diagram itself: you will formalize it to a specific, rigorous standard, and use it to audit your own Part 1 code.

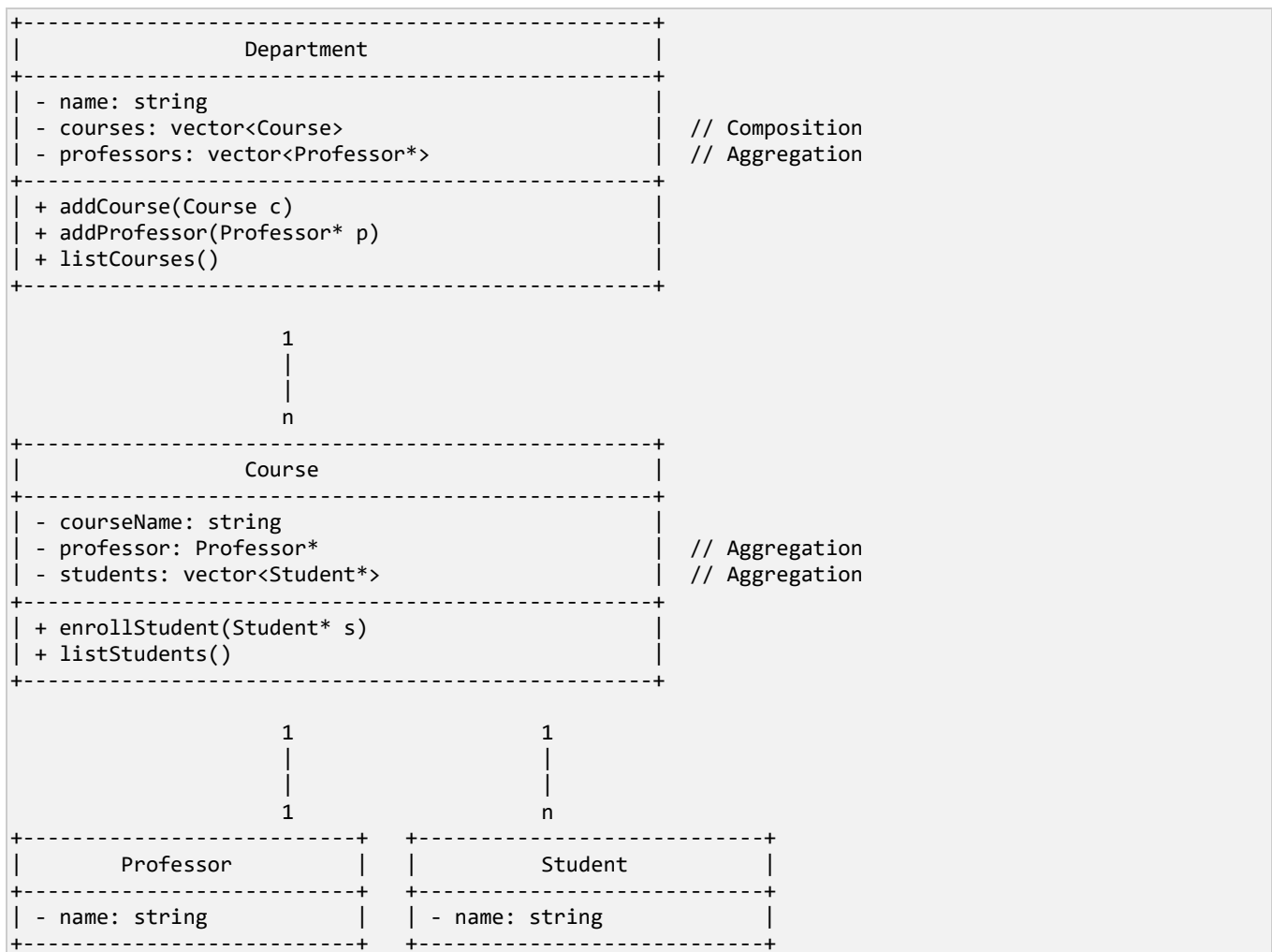
This matters because a class diagram is a communication tool, not a decoration. A reader should be able to tell ownership (composition vs. aggregation) and cardinality (how many of each) from the diagram alone, without reading a line of your `.cpp` file. That's a different skill than getting the code to compile, and it's the one Part 2 is built to practice.

Learning Objectives

- Read and produce standard UML class-diagram notation: visibility markers (`-` private, `+` public), and the filled-diamond / open-diamond convention for composition vs. aggregation.
- Attach multiplicity labels (1, n) to both ends of an association, and connect each label back to the underlying data structure (a single pointer vs. a vector) in your code.
- Diagram from a working implementation, rather than only diagramming before writing code — an equally common and equally important direction to practice.
- Critically evaluate a simplified model: notice where a diagram's multiplicity is more restrictive than real life, and decide whether that's a simplification worth keeping or a bug in the design.

The Reference Sketch

Below is the diagram exactly as given in the lab's starter material. It's deliberately drawn as a plain-text sketch — good enough to convey the idea, but with two rough edges you'll fix in your own version (see "Task 2" below).



Note: the two branches out of Course (to Professor and to Student) are redrawn side-by-side above instead of stacked in a single vertical line. A plain-text sketch can't easily show one box with two separate lines leaving it, so the original source material lists them one after another, which makes it look like the arrow runs from Professor down to Student. It does not — **both** the 1–1 line and the 1–n line start at Course. That's the first thing to fix when you redraw this properly.

Reading the Notation

In the reference sketch	Standard UML symbol	What it means
// Composition comment, Department–Course line	Filled diamond ◆ at the Department end	Department owns its Courses outright; a Course does not outlive its Department.
// Aggregation comment, Course–Professor and Course–Student lines	Open diamond ◇ at the Course end	Course refers to a Professor and some Students, but does not own them; they exist independently.
Stacked 1 / n labels	Multiplicity labels, one at each end of the line	How many instances of each class participate — e.g. 1 Department to n Courses.

Reference API (as sketched)

The field and method names below are taken directly from the sketch. Minor naming differences from your own Part 1 code (for example `courseName` here vs. `title` in your solution) are fine — the diagram is about relationships and ownership, not about matching identifiers exactly.

Department

Member	Purpose
<code>name: string (private)</code>	The department's name.
<code>courses: vector<Course> (private)</code>	Composition. Owns its Courses by value.
<code>professors: vector<Professor*> (private)</code>	Aggregation. References Professors it does not own.
<code>addCourse(Course c)</code>	Adds a course to the department.
<code>addProfessor(Professor* p)</code>	Adds a professor reference to the department.
<code>listCourses()</code>	Prints the department's course catalog.

Course

Member	Purpose
<code>courseName: string (private)</code>	The course's title.
<code>professor: Professor* (private)</code>	Aggregation. References the one Professor teaching this course.
<code>students: vector<Student*> (private)</code>	Aggregation. References the enrolled Students.
<code>enrollStudent(Student* s)</code>	Adds a student to the course's roster.
<code>listStudents()</code>	Prints the course's roster.

Professor / Student

Member	Purpose
<code>name: string (private)</code>	Each class stores only a name in this sketch — same lazy-class shape as Part 1.

Task 1 — Redraw the Diagram Properly

Using a real diagramming tool (draw.io / diagrams.net, Lucidchart, PowerPoint's shapes, or by hand on paper — a photo is fine) produce a class diagram that:

- Draws all four classes as UML class boxes: name, then a divider, then attributes with visibility markers, then a divider, then methods with visibility markers.
- Uses a filled diamond ◆ at the Department end of the Department–Course line, and open diamonds ◇ at the Course end of the Course–Professor and Course–Student lines.
- Puts a multiplicity label at BOTH ends of every line — not just one end, the way the plain-text sketch sometimes does.

- Draws the Course–Professor and Course–Student lines as two separate lines both leaving Course, not as one line running through Professor to reach Student.

Task 2 — Cross-Check Against Your Part 1 Code

Return to the .cpp file you created for Part 1 and carefully verify, field by field and method by method, that it aligns with every aspect of your updated diagram. If your new diagram calls for an accessor that you omitted in Part 1, add it now. If there is a discrepancy between your diagram and code regarding whether a member is a pointer or a value, adjust the diagram—since the code’s ownership rules take precedence and determine whether composition or aggregation is represented.

Task 3 — Reflection Questions

Answer each in two to four sentences, in a short write-up (plain text or a comment block at the top of your .cpp is fine).

1. The reference diagram indicates a 1–1 relationship between Course and Professor. However, the current Course and Department code does not prevent multiple Course objects from referencing the same Professor*. Does this mean the diagram is inaccurate, or is it specifying a rule that the code does not yet enforce? Explain what changes you would make—whether to the diagram, the code, or both—to bring them into alignment.
2. In real life, a professor usually teaches more than one course. If you changed the Course–Professor multiplicity to n–1 (many Courses, one Professor each, but a Professor can appear on more than one Course), would anything in your Course or Department implementation need to change, or does the existing Professor* design already support that?
3. Composition is drawn with a filled diamond only between Department and Course, never between Course and Student, even though a Course "has" Students too. In one or two sentences, say why a Course's relationship to its Students is aggregation and not composition — think about what happens to a Student object when the Course they're enrolled in is destroyed.

Deliverables

1. A redrawn UML class diagram (image, PDF, or hand-drawn photo) satisfying every bullet under Task 1.
2. Written answers to the three Reflection Questions in Task 3.
3. Your Part 1 .cpp file, updated if Task 2 turned up any mismatch between the diagram and the code.

Quality Control

- All four classes appear as proper UML class boxes with visibility markers on every attribute and method.
- Department–Course uses a filled diamond at the Department end; the other three relationships use open diamonds at the Course/Department end.
- Every association line has a multiplicity label at both ends.
- The Course–Professor and Course–Student relationships are drawn as two independent lines out of Course, not as one line passing through Professor.
- All three reflection questions are answered, not just restated.
- Any mismatch found in Task 2 between the diagram and the Part 1 code has been resolved in the submitted .cpp.

Checking Your Work

A solution for this design is available separately as Part 3 in the course repository. It features the same four classes and relationships discussed here, making it a helpful resource to compare with your own diagram and code after submission.